

Ficha nº 1

Programação Funcional em Haskell

Nesta ficha abordam-se sobre os seguintes conceitos da linguagem de programação funcional Haskell:

1. valores e expressões;
2. tipos básicos e tipos compostos;
3. operadores predefinidos;
4. definição de funções simples;
5. cálculo de expressões simples;
6. utilização de módulos;
7. recursividade e funções recursivas.

1 Valores, Expressões e Tipos

Os valores são as entidades básicas da linguagem Haskell. As expressões são obtidas combinando-se valores com funções, operadores (que são também funções) e variáveis.

Exemplos:

valores	Expressões
5	3.8 + 4.6
67.9	True && (not False)
True	((*) 4 ((+) 9 3)) (note os operadores infixos)
'u'	8 * 5 + 4 * ((2 + 3) * 8 - 6)
“abcd”	(toLower (toUpper 'x'))

Os tipos servem para classificar entidades de acordo com as suas características. Em Haskell escreve-se **$e :: T$** para dizer que a expressão **e** *é do tipo* **T** (ou **e** *tem tipo* **T**).

Exemplos:

```
5 :: Int
```

```
67.9 :: Float
```

```
True :: Bool
```

```
'u' :: Char
```

```
(3.8 + 4.6) :: Float
```

```
(True && (not False)) :: Bool
```

```
((*) 4 ((+) 9 3)) :: Int
```

```
(8 * 5 + 4 * ((2 + 3) * 8 - 6)) :: Int
```

```
(toLower (toUpper 'x')) :: Char
```

Tipos Básicos

O Haskell oferece os seguintes tipos básicos:

- **Bool** - Boleanos: True, False
- **Char** - Caracteres: 'a', 'x', 'R', '7', '\n', ...
- **Int** - Inteiros de tamanho limitado: 1, -4, 23467, ...
- **Integer** - Inteiros de tamanho ilimitado: -6, 36, 45763456623443249, ...
- **Float** - Números de vírgula flutuante: 3.5, -45.78, ...
- **Double** - Números de vírgula flutuante de dupla precisão: -45.63, 3.678, 51.2E7, ...
- **()** - Unit: ()

Tipos Compostos

Tuples

$(a_1, a_2, \dots, a_n) :: (T_1, T_2, \dots, T_n)$,

sendo a_1 to tipo T_1 , a_2 do tipo T_2 , ... e a_n do tipo T_n .

Exemplos:

```
(3, 'd') :: (Int,Char)
(True, 5.7, 3) :: (Bool, Float, Int)
('k', (6, 2), False) :: (Char, (Int, Int), Bool)
```

Listas

$[a_1, a_2, \dots, a_n] :: [T]$

Sendo todos os elementos, a_i , da lista do tipo T .

Exemplos:

```
[3, 4, 3, 7, 8, 2, 5] :: [Int]
['r', 'c', 'e', '4', 'd'] :: [Char] (*)
[( 'a', 5), ( 'd', 3), ( 'h', 9)] :: [(Char,Int)]
[[5,6], [3], [9,2,6], [], [1,4]] :: [[Int]]

(*) Notar que ['r', 'c', 'e', '4', 'd'] == "rce4d"
```

Cálculo do Valor de uma Expressão

O interpretador de Haskell ghci usa definições de funções e operadores como *regras de cálculo*, para calcular o valor de uma expressão. Por exemplo, a expressão $8 * 5 + 4 * ((2 + 3) * 8 - 6)$ é calculada pelos seguintes passos:

$8 * 5 + 4 * ((2 + 3) * 8 - 6) \Rightarrow 40 + 4 * (5 * 8 - 6)$

$\Rightarrow 40 + 4 * (40 - 6)$

$\Rightarrow 40 + 4 * 34$

$\Rightarrow 40 + 136$

$\Rightarrow 176$

Tarefa 1

Executar no GHCi:

```
> :set +t
> fst (4, 'a')
> snd (4, 'a')
> fst (5.6, 3)
> :i fst
> :t fst
> :i tail
> :t tail
> tail [6,7,3,9]
> tail "sdferta"
```

Observe o mecanismo de inferência de tipos do Haskell e o polimorfismo das funções `fst` e `tail`.

Tarefa 2

Infira o tipo, se existir, de cada uma das seguintes expressões, e avalie-a. A seguir use o GHCi para verificar a sua inferição.

```
[True, (5>4), (not ('5'=='6')), (True || (False && True))]
((tail "abcdef"),(head "abcdef"))
[(tail "abcdef"),(head "abcdef")]
[4,5,6]++[3,5,8]
(tail [6,7])
concat ["asdf", "bbb", "tyuui", "cccc"]
```

2 Funções: Tipos e Definição

Em Haskell as funções são “first-class citizens”. São objetos com o mesmo estatuto que os valores de tipos básicos e compostos. As funções têm um *tipo*.

Funções

$f :: T1 \rightarrow T2$ - funções que recebem valores do tipo $T1$ e devolvem valores do tipo $T2$.

$(f\ a) :: T2$ - aplicação da função f ao argumento a do tipo $T1$.

Exemplos:

```
toLower :: Char -> Char
not :: Bool -> Bool
ord :: Char -> Int
chr :: Int -> Char
fst :: (a, b) -> a
tail :: [a] -> [a]
```

As funções polimórficas são funções às quais é possível associar mais do que um tipo.

O Haskell recorre a variáveis de tipo ($a, b, c, \dots$) para expressar o tipo de tais funções. Uma variável de tipo representa um tipo qualquer. Quando as funções são usadas, as variáveis de tipo são substituídas pelos tipos concretos adequados.

Um princípio fundamental de qualquer linguagem de programação com tipos é da *compatibilidade do tipo de uma função com os tipos dos seus argumentos*. A verificação desta condição antes da compilação dos programas protege-os da ocorrência de muitos erros durante a sua execução. Para ilustrar o comportamento do interpretador nesta situação, tente avaliar a seguinte expressão:

```
> tail 45
```

Funções pré-definidas

O Haskell oferece um conjunto de funções pré-definidas (cf. o módulo `Prelude`). Alguns exemplos foram introduzidos anteriormente:

- operadores lógicos

&&, ||, not

- operadores relacionais

>, <=, ==

- operadores sobre tuplos

fst, snd

- operadores sobre listas

head, tail, length, reverse, concat, ++

Funções definidas pelo programador

Uma função é definida por uma expressão que relaciona os seus argumentos com o resultado pretendido:

<nomefunção> <arg1>...<argn> = <expressão>

Por exemplo:

```
ex a = 50 * a
funcao1 x y = x + (70*y)
```

Depois de definidas, estas funções podem ser utilizadas para construir novas expressões, que serão avaliadas de acordo com as definições das funções.

Por exemplo:

```
funcao1 (ex 10) 1 ⇒ (ex 10) + (70*1) ⇒ (50*10) + (70*1)
⇒ 500 + (70*1)
⇒ 500 + 70
⇒ 570
```

O tipo de cada função é inferido automaticamente pelo interpretador; no entanto é considerado “boa prática” incluir explicitamente na definição de uma função o seu tipo.

Por exemplo:

uma função que recebe um par de inteiros e devolve como resultado o maior deles. Esta função, a que chamaremos maior, poderá ser definida como se segue:

```
maior :: (Int, Int) -> Int
maior (x,y) = if (x > y) then x else y
```

Para definir uma função que calcule o maior de três inteiros, pode-se usar a função anterior:

```
maiorde3 :: Int -> Int -> Int -> Int
maiorde3 x y z = (maior ((maior (x,y)), z))
```

Note-se que a função “maior” utiliza uma expressão do tipo if/then/else, que se representa da forma:

```
if <expressão booleana> then <expressão 1> else <expressão 2>
```

A expressão if/then/else complete devolve o valor de uma das duas expressões alternativas que se seguem ao then ou ao else: a primeira expressão caso a condição booleana seja verdadeira e a segunda caso a condição seja falsa. Pode-se testar este tipo de expressões através do GHCi de forma fácil, por exemplo:

```
➤ x = 2
➤ y = 3
➤ If x > y then x else y
3
```

Módulos de código Haskell

As definições de funções não podem ser introduzidas diretamente no interpretador de Haskell, devendo antes ser escritas num ficheiro, que se designa por módulo.

Um módulo é um ficheiro de texto contendo um conjunto de definições (declarações de tipos, de funções, de classes, ...) que são lidas pelo interpretador, e depois utilizadas.

Um módulo Haskell é armazenado num ficheiro com extensão `.hs`, `<nome>.hs`, em que `<nome>` representa o nome do módulo, que terá que ser declarado na primeira linha do ficheiro. Por exemplo, o ficheiro `Teste.hs` deverá começar com a declaração seguinte:

```
module Teste where ....
```

Um exemplo de um módulo contendo duas das funções acima definidas será:

```
module Teste where
funcao1 x y = x + (70*y)
ex a = 50 * a
```

Tarefa 3

- (1) Crie um ficheiro com o módulo acima apresentado.
- (2) No GHCi carregue este módulo, escrevendo

```
> :l Teste.hs
```
- (3) Use o interpretador GHCi para verificar qual o tipo das funções definidas no módulo.
- (4) Avalie depois a expressão

```
funcao1 (ex 10) 1.
```

Tarefa 4

1. Crie um módulo com o nome `Circulo` num ficheiro com o nome `Circulo.hs`. Defina as seguintes funções, todas elas recebendo apenas o valor do raio do círculo: `diametro`, `perimetro`, `area`, `areaDaSuperficie`, e `volume`. Carregue este modulo no GHCi e teste as funções definidas.
2. Crie um módulo com o nome `Conversoes` num ficheiro com o nome `Conversoes.hs`. Defina as seguintes funções, todas elas recebendo um argumento e devolvendo o valor correspondente: `fahrenheitToCelsius`, `celsiusToFahrenheit`, `feetToMeters`, `metersToFeet`, `milesToKilometers`, e `kilometersToMiles`. Carregue este modulo no GHCi e teste as funções definidas.
3. Defina e teste as seguintes funções. Crie um módulo com o nome `Ficha1` para incluir as definições seguintes.
 - (1) Defina uma função que receba dois pares de inteiros e retorne um par de inteiros, sendo o primeiro elemento do par resultado a soma dos primeiros elementos dos pares de entrada, e o segundo elemento do par, o produto dos segundos elementos dos pares de entrada.
 - (2) Escreva uma função que, dados três números inteiros, retorne um par contendo no primeiro elemento o maior dos números, e no segundo elemento o segundo maior dos números.
 - (3) Escreva uma função que receba um triplo de números inteiros e retorne um triplo em que os mesmos números estão ordenados por ordem decrescente.
 - (4) Os lados de qualquer triangulo respeitam a seguinte condição: a soma dos comprimentos de quaisquer dois lados, é superior ao comprimento do terceiro lado. Escreva uma função que receba o comprimento de três segmentos de reta e retorne um valor booleano, indicando se satisfazem esta condição.
 - (5) Escreva uma função `abrev` que receba uma string contendo nome de uma pessoa e retorne uma string com o primeiro nome e apelido.

Por exemplo:

```
> abrev "Joao Carlos Martins Sarmento"
> Joao Sarmento
```

As funções, pré-definidas, `words` e `unwords` poderão ser-lhe uteis:

```
words :: String -> [String]
```

Devolve como resultado a lista das palavras (strings) de um texto (uma string)

```
unwords :: [String] -> String
```

constrói um texto (uma string) a partir de uma lista de palavras (strings)

3 Importação de Módulos

Um programa em Haskell é formado por um conjunto de módulos. As definições de cada módulo podem ser utilizadas internamente, ou exportadas para ser utilizadas noutros módulos.

Para utilizar definições contidas num outro módulo é necessário importá-lo explicitamente. Este tipo de ligação entre módulos estabelece-se utilizando a declaração “import”.

Por exemplo, como ter acesso às funções de manipulação de caracteres e *strings* (listas de caracteres) disponíveis no módulo Char.

```
module Conv1 where
    import Data.Char
    con = toLower 'A'
    fun x = toUpper x
```

Uma exceção a esta regra é o módulo Prelude, que constitui a base da linguagem Haskell, e cujas definições estão sempre disponíveis em todos os outros módulos, sem que seja necessário importá-lo.

Tarefa 5

- (1) Crie um ficheiro com o módulo acima apresentado. Use o interpretador GHCi para experimentar a função fun e ver o valor da constante con.
- (2) Crie um ficheiro Exemp.hs com o módulo seguinte:

```
module Exemp where
    import Conv2
    import Data.Char
    conv x = if (isAlpha x) then (upperandlower x) else []
```

e outro ficheiro com o módulo Conv2:

```
module Conv2 where
  import Data.Char
  upperandlower c = [(toLower c), (toUpper c)]
```

Carregue o ficheiro Exemp.hs no ghci e experimente as funções conv e upperandlower.

Verifique qual o tipo das funções.

Tarefa 6

Consulte as definições oferecidas pelo módulo Char, escrevendo

```
> :browse Data.Char
```

4 Introdução às Funções Recursivas

Considere a seguinte definição matemática da função fatorial para números inteiros não negativos:

$$0! = 1$$

$$n! = n * (n-1) * (n-2) * \dots * 1$$

Notando que $n! = n * (n-1)!$, uma possível definição em Haskell da função fatorial, será:

```
fact :: Int -> Int
fact n = if (n==0) then 1 else n * fact (n-1)
```

Repare que esta função é recursiva, i.e. ela aparece na própria expressão que a define. Diz-se também que a função *se invoca a si própria*. O cálculo da função termina porque se atinge sempre o *caso de paragem* ($n=0$).

Tarefa 7

1. Crie um módulo com o nome MyMath e defina as seguintes funções utilizando expressões do tipo if/then/else. Carregue este modulo no GHCi e teste as funções definidas.
 - (1) Defina uma função que calcule o resultado da exponenciação inteira x^y sem recorrer a funções pré-definidas.
 - (2) Defina uma função que devolva o valor do elemento n da sequência de Fibonacci.
2. Adicione ao módulo MyMath funções alternativas que calculam o factorial, exponenciação, e Fibonacci, sem recorrer ao uso de expressões do tipo if/then/else.
3. Uma experiência científica típica é deixar cair uma bola e ver quão alto ela salta. Uma vez que o "salto" da bola é determinado, a proporção fornece um índice de salto. Por exemplo, se uma bola caída de uma altura de 10 metros salta 6 metros de altura, o índice é 0,6 e a distância total percorrida pela bola é 16 metros após um salto. Se a bola continuasse a saltar, a distância após dois saltos seria 10 metros + 6 metros + 6 metros + 3,6 metros = 25,6 metros. Observe que a distância percorrida para cada salto sucessivo é a distância até o chão mais 0,6 dessa distância quando a bola volta para cima. Defina uma função bouncy que espera como argumentos a altura inicial da bola, seu índice de "*bounciness*" e o número de vezes que a bola pode continuar a saltar. A função retorna a distância total percorrida pela bola.

4. Defina e teste as seguintes funções.

- (1) Defina uma função que recebe uma lista constrói o par com o primeiro e o último elemento da lista.
- (2) Defina uma função que dada uma lista, devolve o par com essa lista e com o seu comprimento.
- (3) Defina uma função que dada uma lista de números, calcula a sua média.